

Cuaderno de Ataques - Corte 3

****Clínica Veterinaria (Seguridad, RLS y Caché)****

Sección 1: Tres ataques de SQL injection que fallan

Para probar la seguridad de mi API, intenté vulnerar el buscador de mascotas desde el frontend. Todos los ataques fallaron gracias a que implementé consultas parametrizadas usando la librería pg de Node.js en mi archivo api/index.js.

La línea exacta que defiende al sistema está alrededor de la línea 60:

```
const query = SELECT * FROM mascotas WHERE nombre ILIKE $1
```

Al usar \$1 y pasar los parámetros en un arreglo, la base de datos trata el input estrictamente como un texto a buscar, desactivando cualquier comando SQL malicioso.

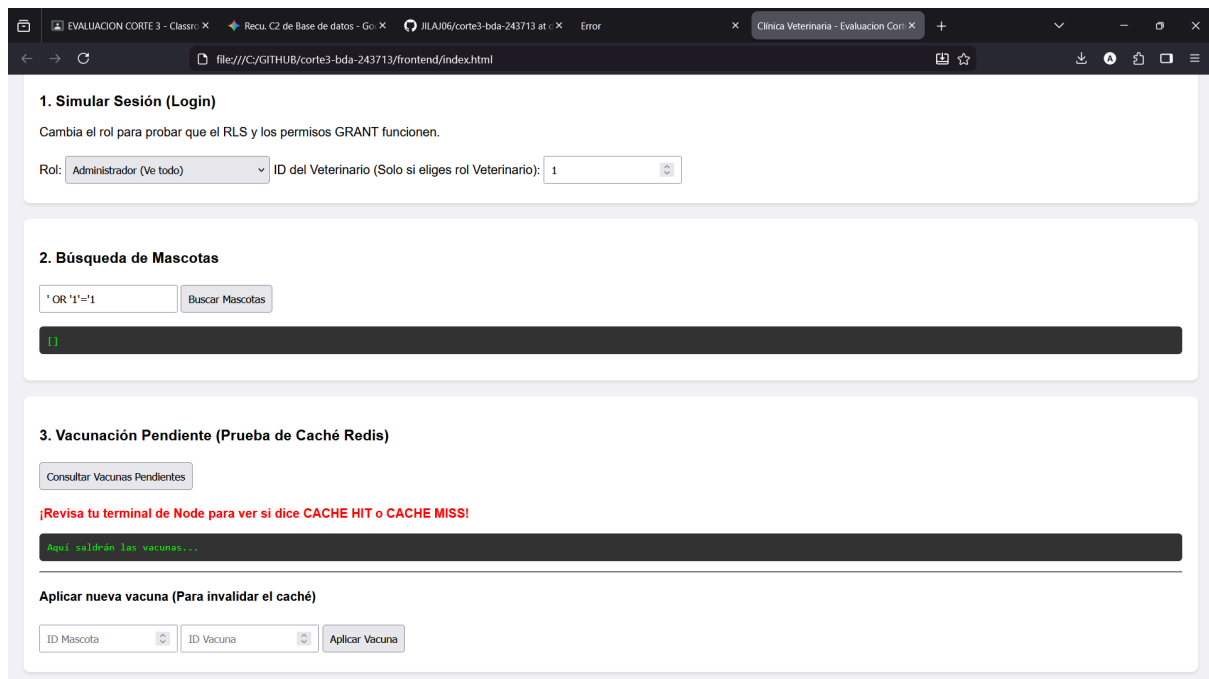
Ataque 1: Quote-escape clásico

*** **Input probado:**** ' OR '1'='1

*** **Pantalla:**** Frontend HTML (Sección de Búsqueda de Mascotas).

*** **Resultado:**** Falló. El sistema devolvió un arreglo vacío [] porque buscó literalmente a una mascota que se llamara así.

*** **Evidencia:****



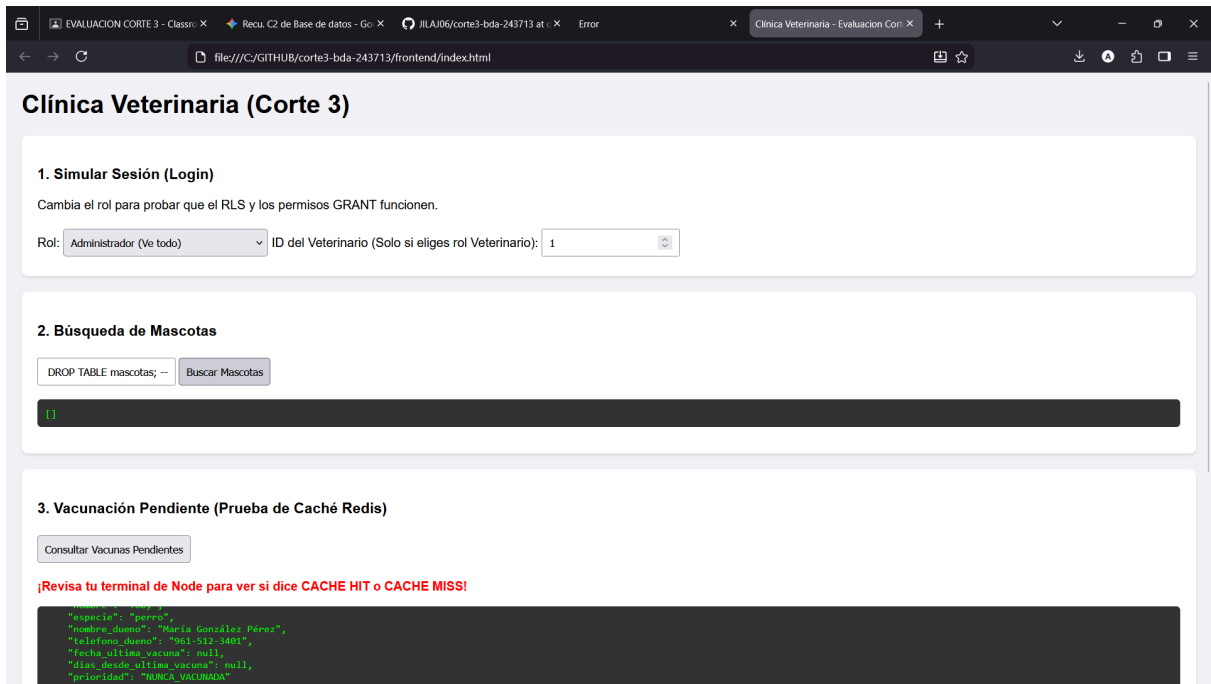
Ataque 2: Stacked query (Intento de borrar tabla)

*** **Input probado:**** '; DROP TABLE mascotas; --

*** **Pantalla:**** Frontend HTML (Sección de Búsqueda de Mascotas).

*** **Resultado:**** Falló. El backend lo interpretó como una cadena de texto inofensiva y devolvió []`.

*** **Evidencia:****



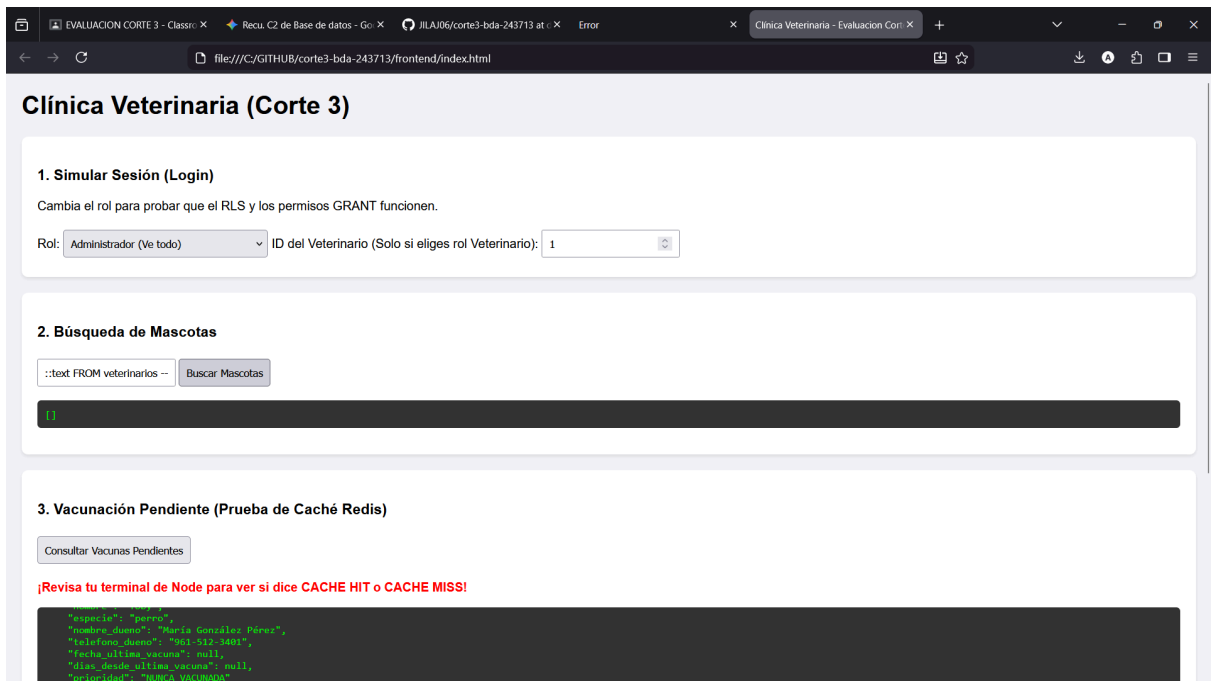
Ataque 3: Union-based (Intento de robo de datos)

* **Input probado:** ' UNION SELECT id, nombre, cedula, dias_descanso, activo::text FROM veterinarios --

* **Pantalla:** Frontend HTML (Sección de Búsqueda de Mascotas).

* **Resultado:** Falló. No se ejecutó la unión de tablas; devolvió [].

* **Evidencia:**

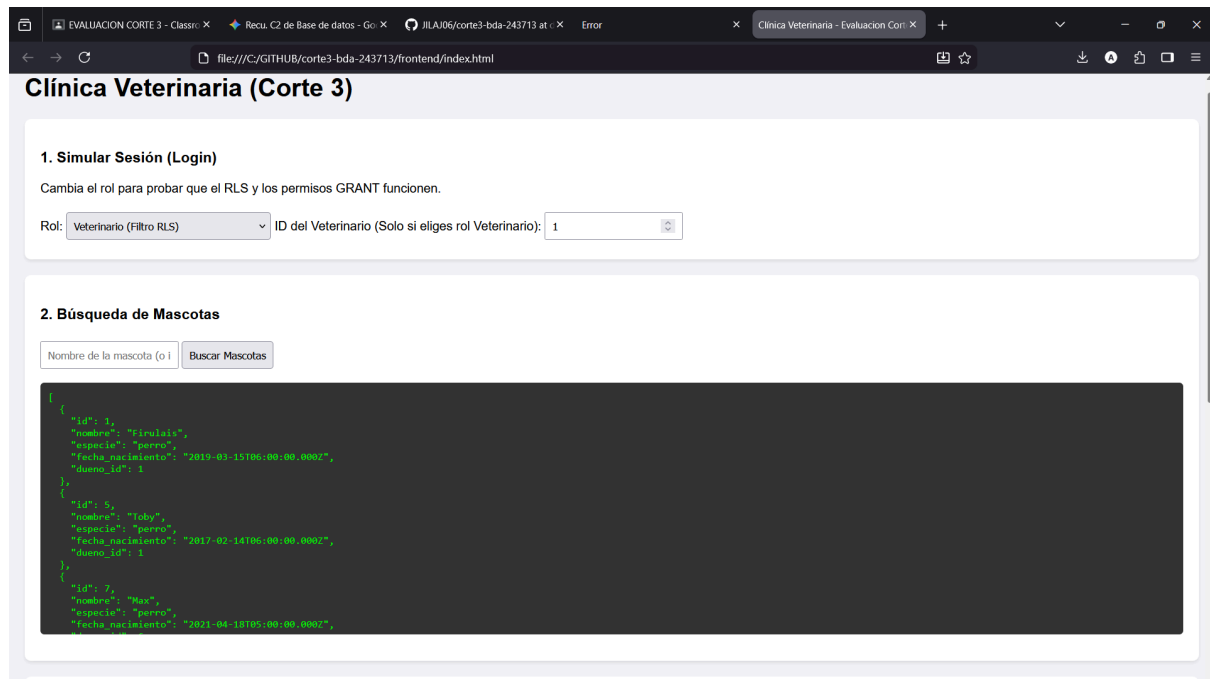


Sección 2: Demostración de RLS en acción

Para esta prueba, utilicé el selector de roles del frontend que simula la autenticación enviando el ID del veterinario a la API.

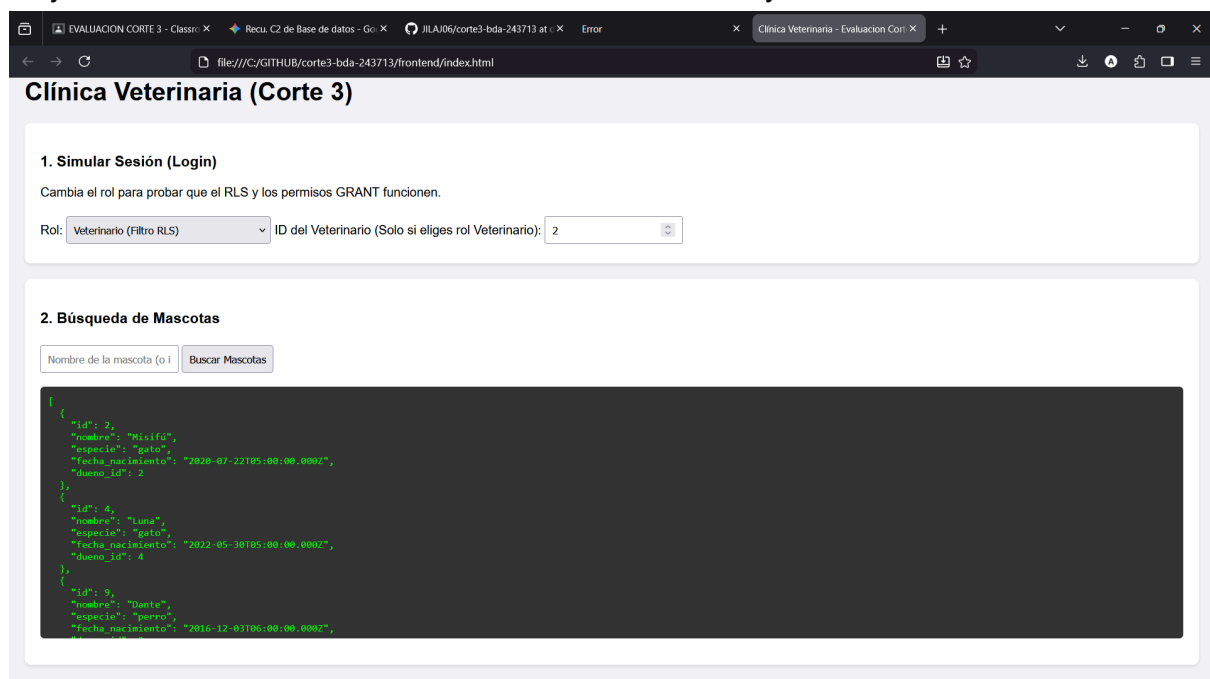
****Prueba con el Veterinario 1 (Dr. López - ID: 1):****

Al buscar todas las mascotas con este rol, el sistema solo me devolvió 3 registros: Firulaís, Toby y Max.



****Prueba con el Veterinario 2 (Dra. García - ID: 2):****

Cambié la sesión a la Dra. García y repetí la búsqueda. Ahora el sistema me arrojó un conjunto totalmente diferente de 3 mascotas: Misifú, Luna y Dante.



****¿Por qué sucede esto?****

Este comportamiento es producto de la política RLS que creé llamada `vet_ve_sus_mascotas`. Esta política intercepta el `SELECT` y filtra las filas obligando a que el ID de la mascota exista en la tabla relacional `vet_atiende_mascota` emparejado exactamente con el ID del veterinario actual (el cual le paso a Postgres mediante `current_setting`).

Sección 3: Demostración de caché Redis funcionando

Implementé Redis en el endpoint de vacunas pendientes para reducir la carga de procesamiento en Postgres.

****Flujo de la prueba y Logs:****

- **Primera consulta (CACHE MISS):**** Consulté la vista por primera vez. El sistema fue hasta PostgreSQL, tardó la latencia normal y me arrojó los datos.
- **Segunda consulta (CACHE HIT):**** Volví a darle clic inmediatamente. El sistema ya no tocó la base de datos; Redis escupió la respuesta en milisegundos.
- **Invalidación (POST):**** Agregué una nueva vacuna desde el formulario. El backend detectó la escritura y mandó borrar todas las llaves asociadas.
- **Tercera consulta (CACHE MISS):**** Al consultar de nuevo, el sistema detectó que el caché estaba vacío y volvió a calcular la vista fresca desde Postgres.

****Evidencia de los Logs en terminal:****

The screenshot shows a VS Code editor with the following components:

- Explorer:** Displays the project structure for `corte3-bda-243713`, including `api`, `backend`, and `frontend` folders.
- Source Code:** The `index.js` file is open, showing the setup for Express, PostgreSQL (using `pg`), and Redis. It includes connection logic and Redis client initialization.
- Terminal:** Shows the execution of `node index.js`. The logs indicate the application is running on `localhost:3001` and show the Redis cache status for a specific query: `[CACHE MISS] consultando a PostgreSQL para la llave: vacunas_pendientes_admin_1`, `[CACHE HIT] vacunacion_pendiente - Llave: vacunas_pendientes_admin_1`, `[CACHE INVALIDADO] Se aplicó una vacuna, caché borrado.`, and `[CACHE MISS] consultando a PostgreSQL para la llave: vacunas_pendientes_admin_1`.
- Right Panel:** Shows the Git status and a commit message: `Agrega un .gitignore para no subir la carpeta node_modules dentro de la carpeta api`.

****Decisiones de Diseño de Redis:****

* **Key utilizada:** Usé el formato dinámico vacunas_pendientes_{rol}_{vetId}. Lo hice así para aislar el caché. Si no separara la llave por ID, el Veterinario 2 podría terminar viendo el caché con los pacientes del Veterinario 1, lo cual rompería el RLS.

* **TTL elegido:** Le asigné 300 segundos (5 minutos) porque es un lapso razonable para revisar un reporte de pendientes en una clínica; no es algo que cambie en tiempo real cada segundo, pero tampoco es seguro dejarlo horas sin actualizarse.